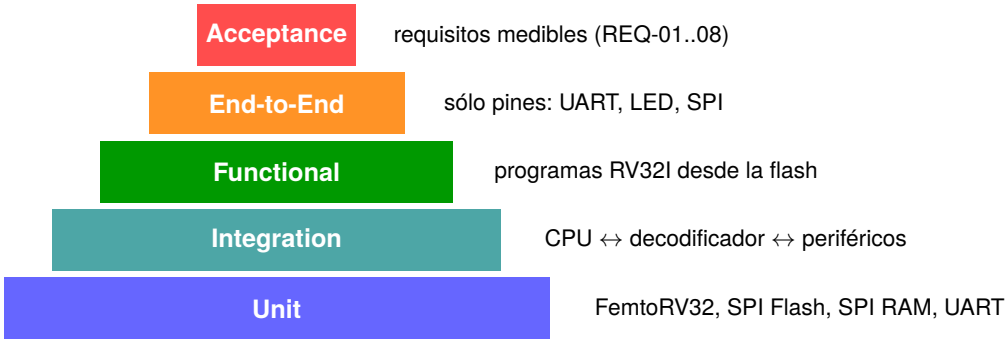
Verificación del silicio `femto_UN`

Núcleo RISC-V FemtoRV32 “Quark” + controladores SPI Flash / SPI RAM + UART

Tiny Tapeout — RTL de referencia: commit `f15fccd`



74 / 98 pruebas pasan con el emulador de memoria | **32 / 98** con memorias comerciales

0 / 45 pruebas a nivel SoC pasan con una flash y una RAM SPI estándar

5 / 5 pruebas de co-simulación del emulador RTL pasan (incl. C compilado con GCC)

Índice general

1 Resumen ejecutivo **2**

2 Alcance, entorno y estrategia **2**
2.1 Qué se verificó y contra qué RTL . . . 2
2.2 Los cinco niveles de prueba . . . 3
2.3 Dos perfiles de memoria externa . . . 4
2.4 Garantías de calidad del propio banco de pruebas . . . 4

3 Arquitectura del chip bajo prueba **4**

4 Resultados del núcleo FemtoRV32 Quark (todas las instrucciones) **5**
4.1 Cobertura de instrucciones . . . 5
4.2 Latencia por instrucción y mutaciones . . . 6

5 Defectos encontrados en el silicio **6**
5.1 DEF-R1: la primera instrucción tras el reset nunca se ejecuta . . . 6
5.2 DEF-F1 y DEF-S1: el primer bit del comando se pierde (depende de la fase) . . . 7
5.3 DEF-F2: los datos de la flash llegan desplazados un bit . . . 9
5.4 DEF-S2: la SPI RAM sólo devuelve 31 bits . . . 9
5.5 DEF-S3: SB, SH, LB, LH, LBU y LHU sobre la SPI RAM son incorrectos . . . 10
5.6 DEF-U1 y DEF-U2: UART . . . 10
5.7 Otros hallazgos . . . 11

6 Cómo hacer que funcione el RISC-V en este silicio **11**
6.1 Diagnóstico: por qué no arranca con una flash normal . . . 11
6.2 Solución de hardware: emulador de memoria `silicon_spi_mem` . . . 11
6.3 Alternativas evaluadas . . . 12
6.4 Reglas de firmware (verificadas por las pruebas) . . . 13
6.5 Compilar C para el chip . . . 13
6.6 Lista de verificación para el *bring-up* . . . 13

7 Aceptación de requisitos **14**

8 Limitaciones del análisis y trabajo futuro **14**

A Cómo ejecutar la suite **14**

B Resultado de cada prueba **14**

C RTL del emulador de memoria **16**

D Arranque y ejemplo en C **17**

1 Resumen ejecutivo

Este informe documenta la verificación completa del RTL del silicio ya fabricado (commit `f15fccd`, anterior al arreglo de `MappedSPIFlash`). No se modificó ningún archivo del diseño: las pruebas se ejecutan contra el RTL tal cual está en el chip. Se construyeron **98 pruebas automatizadas** en cinco niveles (Unit, Integration, Functional, End-to-End y Acceptance) más **5 pruebas de co-simulación** de una solución de hardware.

Conclusión principal

- **El núcleo RISC-V es correcto.** Las 37 instrucciones RV32I del núcleo (más `RDCYCLE`) pasan 20/20 pruebas unitarias, incluidos 24 programas aleatorios (6328 instrucciones) contra un simulador de referencia.
- **El chip, conectado a una flash y una RAM SPI comerciales, no arranca** (0/45 pruebas a nivel SoC). Dos defectos del controlador `MappedSPIFlash`: el primer bit del comando se pierde en 1/3 de las fases del divisor y los datos llegan desplazados un bit. Además, la primera lectura tras el reset se pierde.
- **Sí se puede hacer funcionar el RISC-V sin modificar el silicio**: sustituyendo las memorias por un emulador "consciente del silicio" (FPGA/CPLD, RTL incluido y verificado en co-simulación) y siguiendo 8 reglas de firmware. Con eso el chip arranca, ejecuta el autotest de ISA, imprime por UART y corre código C compilado con GCC.
- Defectos residuales que ninguna memoria externa puede corregir: la SPI RAM sólo guarda 31 bits fiables, no soporta `SB/SH/LB/LH` y la UART sólo da 115200 baudios con reloj de ≈24 MHz (no con 27 MHz).

2 Alcance, entorno y estrategia

2.1 Qué se verificó y contra qué RTL

- **RTL:** commit `f15fccd` (silicio). Los archivos `femtorv32_quark.v` y `MappedSPIRAM.v` son idénticos a los de `baab83f`; `MappedSPIFlash.v` es la versión *anterior* al arreglo posterior (`b28ff83`). Nada se corrigió: los defectos se documentan y se compensan fuera del chip.
- **Herramientas:** Icarus Verilog 12, cocotb 2.1, Python 3.12; GCC `riscv64-unknown-elf` 13.2 para el firmware C.
- **Módulos bajo prueba:** `femtorv32_quark.v` (todas las instrucciones), `MappedSPIFlash.v`, `MappedSPIRAM.v`, además de `perip_uart.v/uart.v`, el decodificador de `femto.v` y el chip completo `tt_um_femto`.

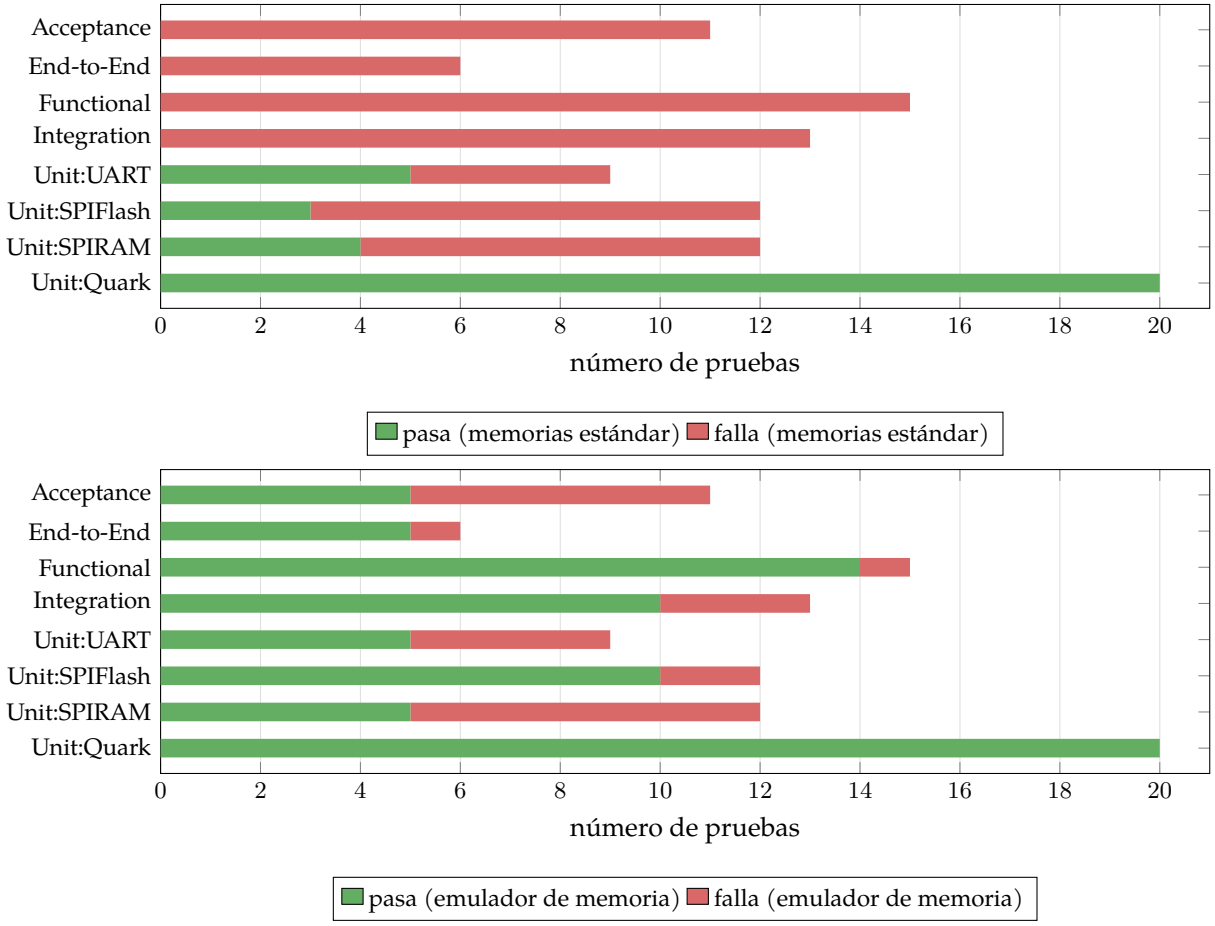


Figura 1: Resultados por nivel de prueba. Arriba: flash y RAM SPI comerciales (modelos conformes al datasheet). Abajo: con el emulador `silicon_aware`. Los fallos restantes son defectos propios del silicio (catálogo en la sección 5); la prueba de aceptación REQ-08 usa memorias estándar siempre y por eso falla en ambos.

2.2 Los cinco niveles de prueba

Nivel	DUT (cocotb toplevel)	Qué comprueba	N°
Unit	`FemtoRV32`, `MappedSPIRAM`, `MappedSPIFlash`, `peripheral_uart`	Cada módulo aislado contra su especificación: instrucciones, protocolo SPI, fases del divisor, UART	53
Integration	`tt_um_femto`	Interfaces CPU ↔ decodificador ↔ periféricos; busy/handshake; mapa de memoria	13
Functional	`tt_um_femto`	Programas RV32I desde la flash; cada función (ALU, saltos, loads/stores, UART, LED, pila) contra un valor de referencia	15
End-to-End	`tt_um_femto`	Chip visto *sólo por sus pines*: banner por UART, eco, LED, reinicio, autotest de ISA completo	6
Acceptance	`tt_um_femto`	8 requisitos medibles (REQ-01..08): arranque, ISA, memoria, velocidad, baud TX/RX, robustez, memorias comerciales	11

Cuadro 1: Jerarquía de pruebas. Total: 98 pruebas + 5 de co-simulación de hardware.

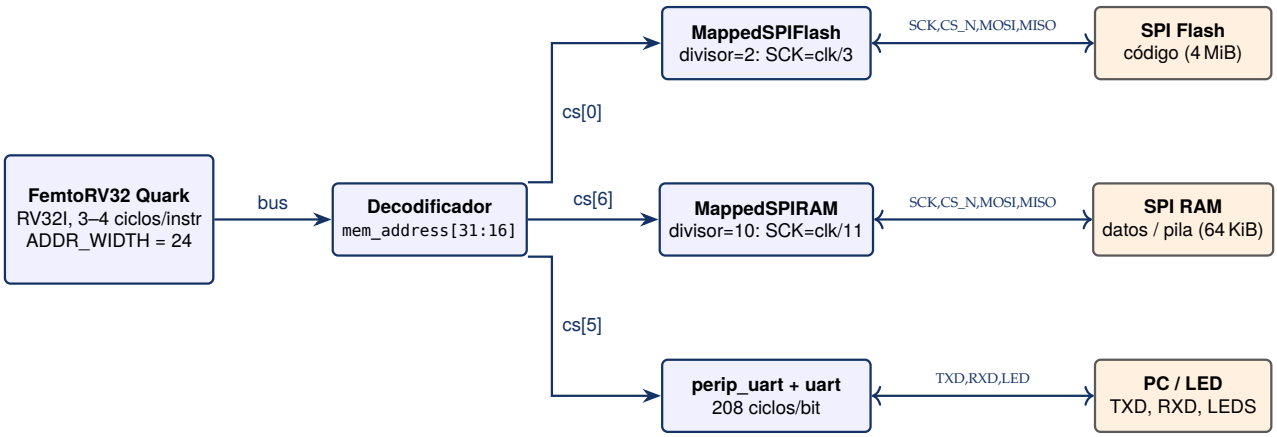


Figura 2: Diagrama de bloques de `tt_um_femto`. No hay memoria interna: todo el código y los datos pasan por los dos controladores SPI. Los bloques naranjas están fuera del chip (placa).

2.3 Dos perfiles de memoria externa

El chip no tiene RAM interna: el código viene de una flash SPI y los datos de una RAM SPI externas. Por eso *lo que haya al otro lado del bus SPI forma parte del experimento*. Todas las pruebas se ejecutan con dos modelos del lado de la memoria:

`standard` Memoria comercial conforme al datasheet (W25Qxx / 23LCxxx / FM25Vxx), SPI modo 0: muestrea MOSI en el flanco de subida, cambia MISO en el de bajada, `READ=0x03`; un comando desconocido se ignora. Es lo que ocurriría al soldar chips normales.

`silicon_aware` Emulador que conoce los defectos del silicio y los compensa desde la memoria (sección 6). Existe como modelo Python (para correr toda la jerarquía) y como RTL sintetizable (`silicon_spi_mem.v`, co-simulado).

Una prueba que falla con `standard` y pasa con `silicon_aware` identifica un defecto *corregible desde la placa*; una que falla en ambos identifica un defecto *intrínseco del silicio*.

2.4 Garantías de calidad del propio banco de pruebas

- **Pruebas de mutación del núcleo:** se inyectaron 6 errores en el RTL (SRA→SRL, BLT→BLTU, LB sin signo, JALR sin limpiar el bit 0, máscara de SH, desbordamiento en SLT); las 6 fueron detectadas (Tabla 4).
- **Referencia independiente:** simulador de instrucciones (ISS) con decodificación propia, distinto del ensamblador y del RTL.
- **Fallo rápido y explicado:** si la CPU no logra leer su primera instrucción, cada prueba SoC falla de inmediato con la causa (*"la CPU no arranca: comando visto por la flash=0x6"*) en lugar de agotar el tiempo.
- **Estado inicial indeterminado:** registros y contador `cycles` arrancan con valores aleatorios, como en silicio real.

3 Arquitectura del chip bajo prueba

Rango (`mem_address`)	Destino	Notas
`0x0000_0000 -- 0x0000_FFFF`	SPI Flash	código; el reset empieza en `0x0`
`0x0001_0000 -- 0x0001_FFFF`	SPI RAM	`word_address = mem_address[15:0]` (dirección de bytes)
`0x0040_0000`	UART	`+0x08` dato; `+0x10` control/estado (bit2 LED, bit3 tx_wr; bit9 tx_busy)
resto	SPI Flash (alias)	bits [21:2] → 4 MiB; `mem_address` tiene 24 bits (alias cada 16 MiB)

Cuadro 2: Mapa de memoria visto por la CPU. Pines: `uo_out`[0]=MOSI flash, [1]=MOSI RAM, [2]=CS_N flash, [3]=CS_N RAM, [4]=SCK RAM, [5]=SCK flash, [6]=LED, [7]=TXD; `ui_in`[0]=MISO flash, [1]=MISO RAM, [2]=RXD.

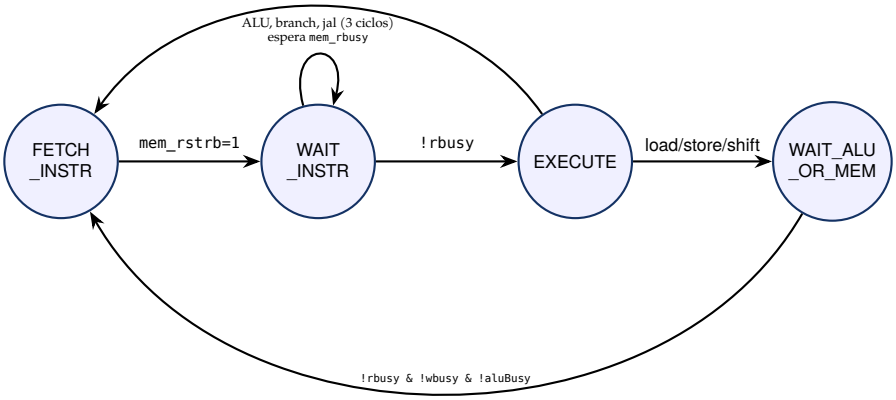


Figura 3: Máquina de estados del núcleo (codificación one-hot). Medido: ALU/branch/jal = 3 ciclos; load/store = 4; desplazamiento de s bits = $s + 4$. Con la flash SPI en el camino cada instrucción cuesta ≈198 ciclos.

4 Resultados del núcleo FemtoRV32 Quark (todas las instrucciones)

El núcleo se probó con una memoria ideal (modelo de memoria con estados de espera aleatorios), sin SPI de por medio. **20 de 20 pruebas unitarias pasan.**

4.1 Cobertura de instrucciones

Grupo	Instrucciones	Prueba(s)
Constantes	LUI, AUIPC	`lui_auipc`, f06, autotest F
Saltos	JAL, JALR (bit 0 forzado a 0, enlace PC+4)	`jal`, `jalr`, f05, autotest E
Branches	BEQ, BNE, BLT, BGE, BLTU, BGEU (tomado y no tomado; hacia atrás, ±4 KiB)	`branches_all`, `far_offsets`, f04, autotest D
Loads	LB, LBU, LH, LHU, LW (todos los desplazamientos, offsets negativos)	`load_store_all_widths`, f07, autotest G
Stores	SB, SH, SW (carriles de byte)	`store_widths_and_lanes`, autotest H–I
ALU imm	ADDI, SLTI, SLTIU, XORI, ORI, ANDI	`alu_imm_all_ops` (1008 casos), f02, autotest B
ALU reg	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND	`alu_reg_all_ops` (1210 casos), f01, autotest A
Desplazamientos	SLLI, SRLI, SRAI y variantes registro, los 32 valores	`shifts_all_amounts` (768 casos), f03, autotest C
Sistema	RDCYCLE; ECALL/EBREAK como NOP; x0 fijo a 0	`rdcycle_and_system`, `x0_hardwired_zero`

Cuadro 3: Cobertura de las 37 instrucciones RV32I del núcleo (más RDCYCLE). `FENCE` no está soportada (decodifica como JAL).

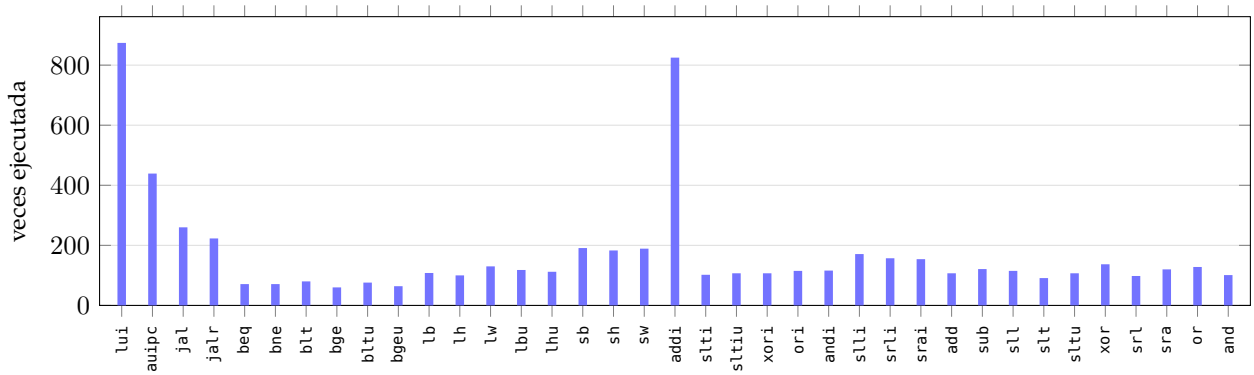
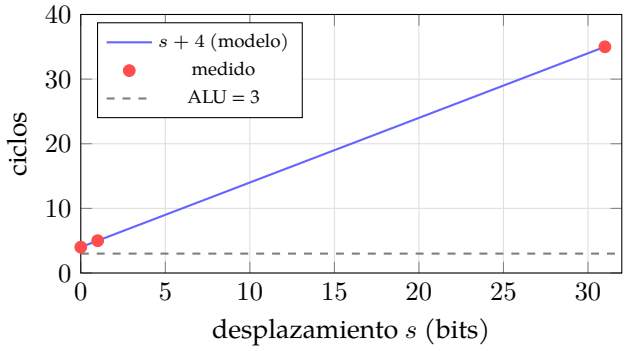


Figura 4: Instrucciones ejecutadas en los 24 programas aleatorios diferenciales (6328 instrucciones en total). Las 37 aparecen al menos 60 veces; la traza de PCs, los 32 registros y la memoria coinciden con el ISS en todas las ejecuciones.

4.2 Latencia por instrucción y mutaciones



Instrucción	Ciclos
ALU / branch / JAL / JALR	3
LW / SW (memoria ideal)	4
SLLI/SRLI/SRAI s bits	$s + 4$
con flash SPI (cualquier)	≈198
LW/SW a SPI RAM	≈700

Figura 5: Ciclos por instrucción medidos con `RDCYCLE`. Los desplazamientos son iterativos (1 bit por ciclo).

Mutación inyectada	Instrucción afectada	Detectada	Pruebas que fallan
SRA → SRL (relleno de signo)	SRA/SRAI	✓	alu_reg, shifts, shift_rd, aleatorias ×2
BLT usa comparación sin signo	BLT	✓	branches, aleatorias ×2
LB sin extensión de signo	LB	✓	load_store, aleatorias ×2
JALR no limpia el bit 0	JALR	✓	jalr, aleatorias ×2
Máscara de SH fija en 0011	SH	✓	store_widths, aleatorias ×2
SLT ignora el desbordamiento	SLT/SLTI/B*	✓	alu_reg, alu_imm, branches, loops, aleat. ×2

Cuadro 4: Pruebas de mutación del núcleo: 6/6 detectadas.

5 Defectos encontrados en el silicio

ID	Módulo	Severidad	Descripción	Pruebas
DEF-F1	MappedSPIFlash	Crítico	El primer bit del comando SPI se pierde en 1 de cada 3 fases del divisor	3
DEF-F2	MappedSPIFlash	Crítico	Los datos de lectura llegan desplazados 1 bit (D31 se pierde, el LSB es el MSB de la palabra siguiente)	8
DEF-R1	SoC / reset	Alto	La primera lectura tras reset se pierde: la instrucción de la dirección 0 nunca se ejecuta	1
DEF-S1	MappedSPIRAM	Crítico	Se pierde el primer bit del comando SPI en 4 de cada 11 fases (lecturas y escrituras)	6
DEF-S2	MappedSPIRAM	Alto	La lectura captura 31 bits: el bit 31 de rdata es el LSB de la lectura anterior	8
DEF-S3	MappedSPIRAM / SoC	Alto	SB/SH/LB/LH/LBU/LHU sobre SPI RAM son incorrectos	4
DEF-C1	SPI (ambos)	Medio	CS_N y SCK cambian en el mismo instante en algunas fases (setup 0 ns)	2
DEF-U1	uart	Alto	Baud real = f_clk/208: sólo es 115200 con f_clk ≈ 23.96 MHz (a 27 MHz es 129.8 kbaud, +12.7 %)	4
DEF-U2	uart	Medio	El receptor evalúa el bit de parada un bit tarde: no detecta stop erróneo y corrompe tramas consecutivas	3
INFO-A1	femtorv32_quark	Info	ADDR_WIDTH=24: AUIPC y direcciones load/store se truncan a 24 bits (alias cada 16 MiB)	–
INFO-A2	femtorv32_quark	Info	FENCE no soportado; ECALL/EBREAK/SYSTEM = RDCYCLE; sólo accesos alineados	–
INFO-A3	SoC	Info	Direcciones sin mapear hacen alias a la flash; registros y 'cycles' arrancan con valores indeterminados	–

Cuadro 5: Catálogo de defectos. "Pruebas" = número de pruebas que lo exponen. Todos están tipificados en `verif/tools/defects.py`.

5.1 DEF-R1: la primera instrucción tras el reset nunca se ejecuta

Tras soltar el reset, el núcleo pasa a `FETCH_INSTR` en el primer ciclo y emite `mem_rstrb`. En ese mismo ciclo el controlador de flash sigue en su estado `START` (necesita un ciclo para llegar a `WAIT_STRB`), así que ignora el strobe; `rbusy` nunca sube y la CPU latchea `rdata`=0: ejecuta la instrucción fantasma `0x00000000` (un `lb`

`x0,0(x0)` inofensivo). Se comprobó: con `addi x5,x0,11` en la dirección 0, `x5` conserva su valor aleatorio de arranque mientras que `x6..x8` (direcciones 4, 8, 12) son correctos. **Regla R1: poner un NOP en la dirección 0.**

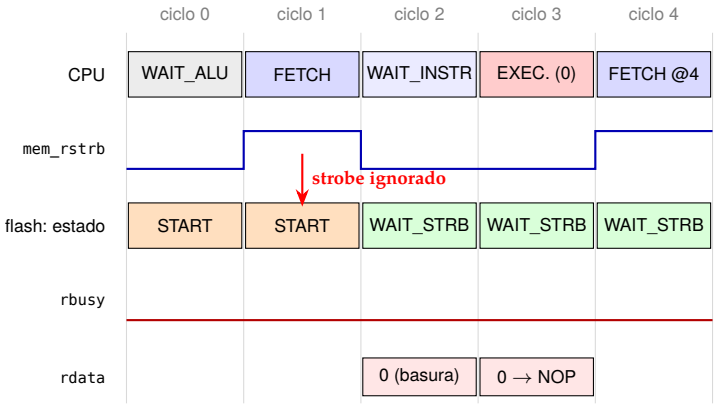


Figura 6: DEF-R1 (resultado de la simulación del chip completo): el primer `mem_rstrb` coincide con el estado `START` del controlador de flash.

5.2 DEF-F1 y DEF-S1: el primer bit del comando se pierde (depende de la fase)

Ambos controladores generan SCK con un contador *libre* (`div_counter`) y desplazan MOSI en un "tick" un ciclo después de cada flanco de bajada. Pero `CS_N` baja en cuanto llega `rd/rstrb`, en *cualquier* fase. Si el primer tick llega antes del primer flanco de subida de SCK, el primer bit del comando se descarta: la memoria recibe `0x06` en lugar de `0x03` (`0x04` en vez de `0x02` al escribir) y la dirección queda desplazada (Fig. 7).

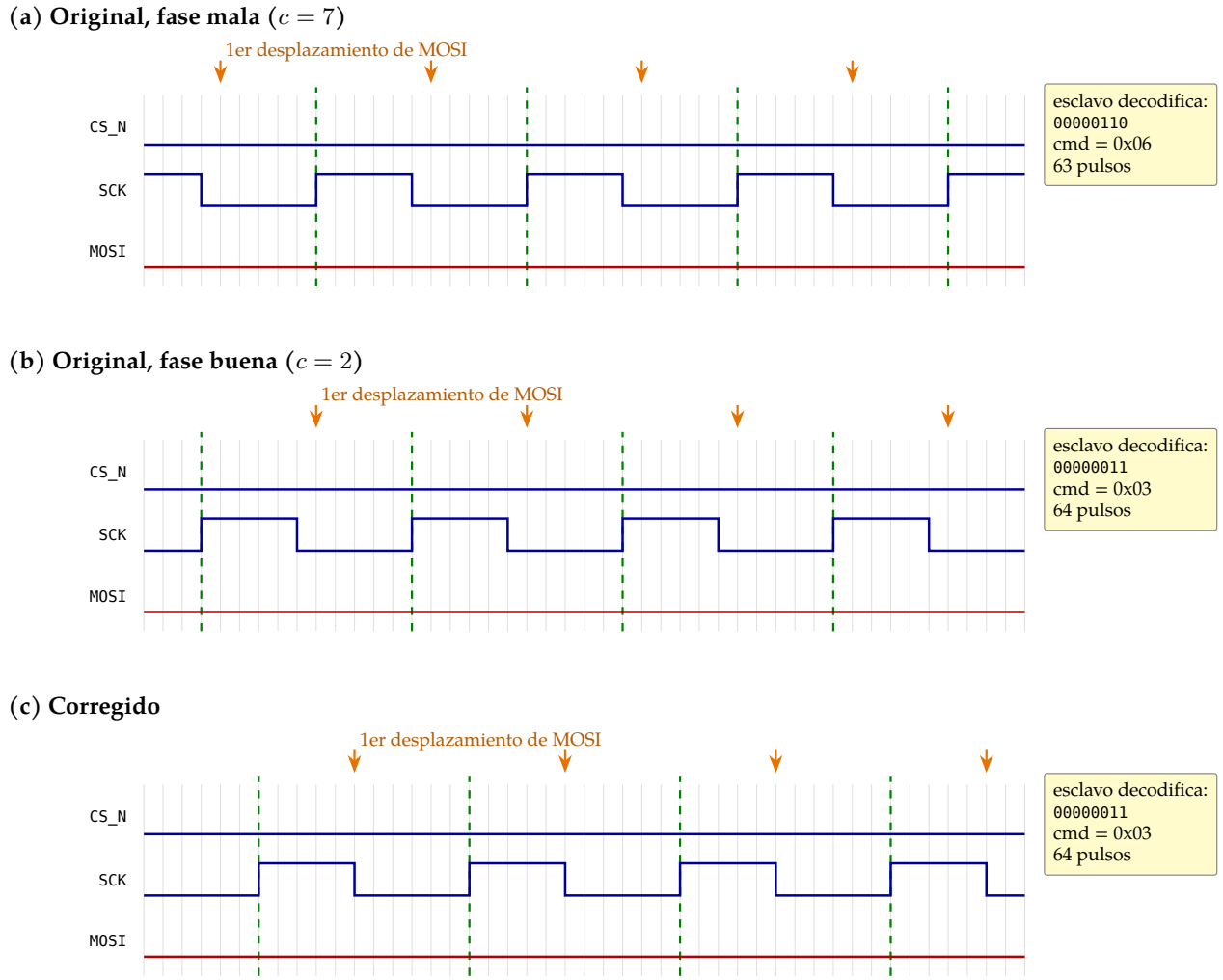


Figura 7: Inicio de una lectura en la SPI RAM (formas de onda reales, 1 muestra por ciclo de reloj). Las líneas verdes son los flancos de subida (donde la memoria muestrea); las flechas naranja, cada desplazamiento de MOSI. (a) Fase mala: el primer desplazamiento ocurre antes de cualquier flanco de subida; la memoria decodifica `0x06`. (b) Fase buena. (c) Controlador corregido (propuesta, no es el silicio).

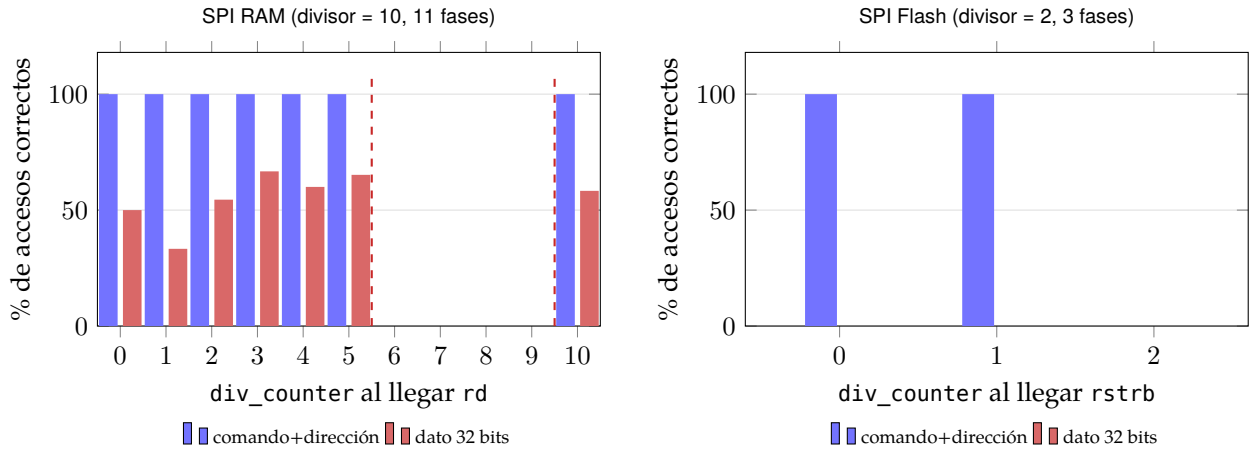


Figura 8: Fracción de accesos correctos según la fase del divisor (240 lecturas y 120 lecturas de flash, memoria estándar). RAM: las fases 6–9 (4 de 11) corrompen el comando el 100 % de las veces; incluso en fases buenas sólo el 37.5 % de las lecturas devuelve el dato completo y el 56.7 % de las escrituras queda correcto. Flash: la fase 2 (1 de 3) pierde el comando; el dato está mal en *todas* las fases.

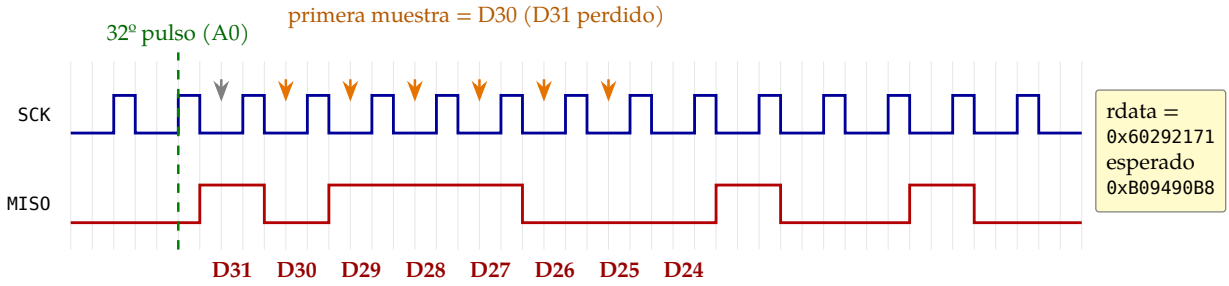
Por qué en la flash es determinista y fatal

El periodo de SCK de la flash es 3 ciclos y una instrucción completa cuesta 195 ciclos (un múltiplo de 3). Por tanto la fase de cada *fetch* respecto al divisor es siempre la misma que la del primero tras el reset. En la simulación esa fase es la mala: **todas** las lecturas llegan con el comando `0x06` (Write Enable) y la CPU ejecuta ceros para siempre.

5.3 DEF-F2: los datos de la flash llegan desplazados un bit

El maestro muestrea MISO *un ciclo después* del flanco de bajada de SCK, cuando una flash real ya cambió al bit siguiente ($tV \approx 7\,\text{ns} \ll 1$ ciclo). La primera muestra de datos es D30 en lugar de D31: la palabra se desplaza un bit y el LSB es el MSB de la palabra siguiente (Fig. 9). Con el mismo hardware, un emulador que presenta D31 una caída después (1 ciclo *dummy*) entrega la palabra exacta.

(a) Flash estándar (W25Q): el dato D31 aparece en la caída 32, que el chip no muestrea



(b) Emulador `silicon_aware`: D31 se presenta una caída después (1 ciclo dummy)

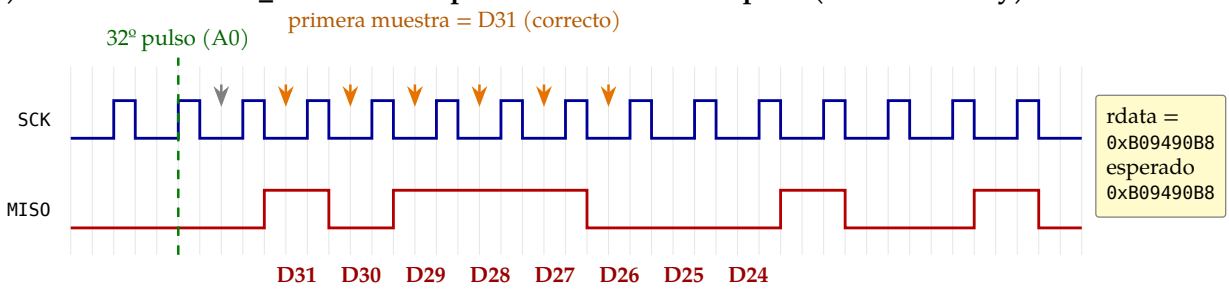
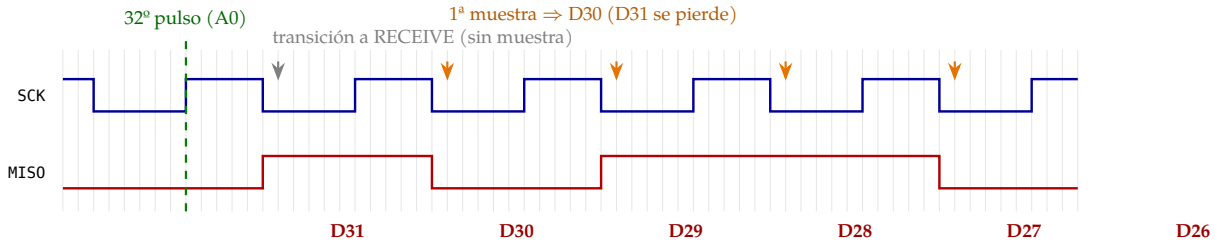


Figura 9: Lectura de la palabra `0xB09490B8` desde la flash del silicio (RTL real). Arriba: memoria estándar ⇒ el chip recibe `0x60292171`. Abajo: emulador con 1 ciclo dummy ⇒ `0xB09490B8`. Las flechas naranja son los instantes de muestreo del chip.

5.4 DEF-S2: la SPI RAM sólo devuelve 31 bits

El controlador de la RAM captura *31* muestras (no 32): `rcv_bitcount` termina con `<= 1` y `rcv_data` no se limpia. Resultado: el bit 31 de cada lectura es el *último bit de la lectura anterior*. Ejemplo medido: tras leer `0x00000001`, leer `0x00000000` devuelve `0x80000000`. No se puede corregir desde fuera: son 31 bits fiables por palabra.

(a) Original: el maestro muestrea un ciclo después del flanco de bajada



(b) Corregido: el maestro muestrea en el flanco de subida

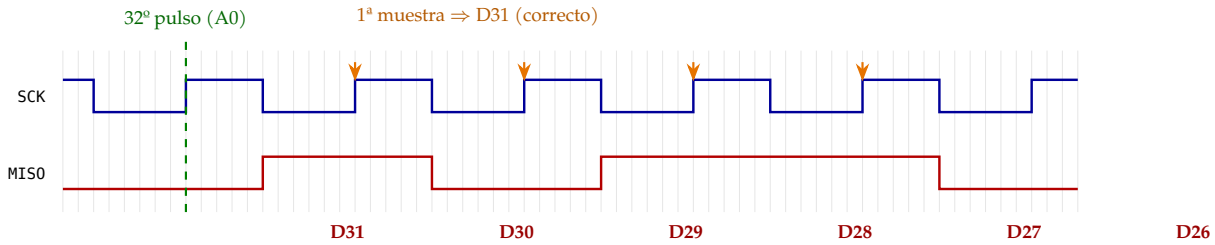


Figura 10: Fase de datos en la SPI RAM. (a) El maestro sólo toma 31 muestras (D30...D0) y D31 nunca se captura. (b) Controlador corregido (propuesta).



Figura 11: Lectura de `0xB09490B8` devuelve `0x309490B8`: el bit 31 es el LSB de la lectura anterior.

5.5 DEF-S3: SB, SH, LB, LH, LBU y LHU sobre la SPI RAM son incorrectos

Dos causas combinadas: (i) `MappedSPIRAM` ignora `mem_wmask`: siempre escribe 32 bits, y para un `sb` el bus lleva el byte replicado 4 veces; (ii) envía la dirección de *bytes* sin alinear, de modo que `lb`/`lh` reciben la palabra desplazada. Un `sb` a `A+1` escribe 4 copias en `A+1..A+4`, corrompiendo también el primer byte de la palabra siguiente. Es la causa de los grupos "SB/SH" y "LB/LH/LBU/LHU (RAM)" del autotest (`ISA PPPPPPPPFF`).

5.6 DEF-U1 y DEF-U2: UART

`uart.v` calcula `divisor = freq/baud/16 = 14` (truncado) y recarga el contador con `divisor-1`: el periodo del *tick* es de 13 ciclos y cada bit dura **208 ciclos**. El baud real es $f_{clk}/208$: exacto a 23,96 MHz, pero $+12{,}7\,\%$ con los 27 MHz del parámetro de diseño (Fig. 12). Además el receptor evalúa el bit de parada con *un bit de retraso* (`rx_bitcount==10`): no detecta un stop erróneo y destruye tramas consecutivas.

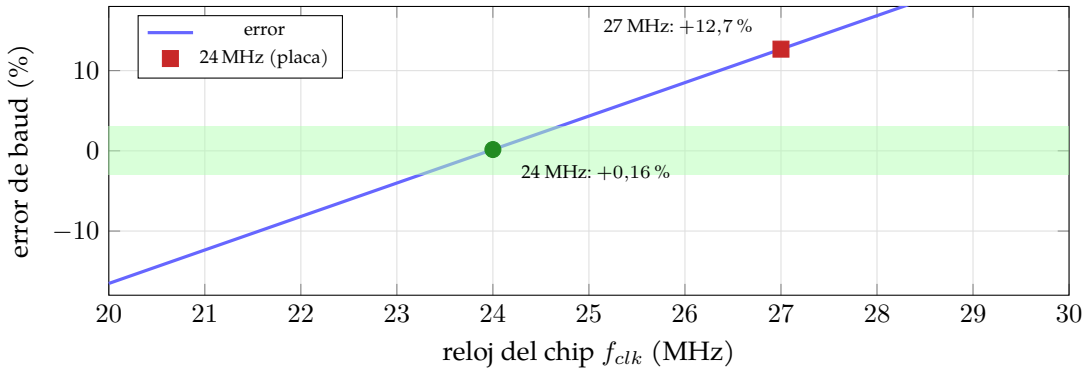


Figura 12: Error del baud real respecto a 115200 en función del reloj. La banda verde es ±3 %. Medido: 208 ciclos/bit.

5.7 Otros hallazgos

- **DEF-C1:** en algunas fases `CS_N` y el primer flanco de SCK (o una caída) cambian en el mismo instante (setup de 0 ns). Una flash real exige tCSS ≥ 5–10 ns. El emulador lo tolera.
- **DEF-S2 en la pila:** los registros que GCC guarda/restaura en la pila (SPI RAM) vuelven con el bit 31 aleatorio; un `fib(n)` recursivo se cuelga en simulación. De ahí la regla R5.
- **INFO:** `ADDR_WIDTH=24` (alias cada 16 MiB), `FENCE` no soportada, registros y `cycles` sin reset, direcciones fuera de mapa con alias a la flash.

6 Cómo hacer que funcione el RISC-V en este silicio

El silicio no se puede cambiar, pero casi todos los defectos se pueden *compensar desde fuera* (la memoria) y desde el software (el firmware). Esta sección es la receta, y cada afirmación está respaldada por una prueba.

6.1 Diagnóstico: por qué no arranca con una flash normal

1. **Comando corrupto (DEF-F1).** En la fase real del chip, el primer fetch ya llega con `0x06`: la flash lo interpreta como *Write Enable* y no devuelve datos.
2. **Aun sin eso, dato desplazado (DEF-F2).** Una flash estándar entrega `0x60292171` donde el código dice `0xB09490B8`.
3. **La instrucción 0 se pierde (DEF-R1).**

Pre-desplazar la imagen de la flash no basta: corregir el bit corrido es posible por software (desplazar toda la imagen un bit), pero el comando corrupto (1) no, porque ocurre en cada acceso con una fase que depende de los ciclos transcurridos.

6.2 Solución de hardware: emulador de memoria `silicon_spi_mem`

Se coloca en la placa, en lugar de la flash y la RAM comerciales, un pequeño módulo digital (FPGA/CPLD) que usa **el mismo reloj** que el chip. El archivo `verif/hw/silicon_spi_mem.v` (sección C) es sintetizable (arrays de bytes inferidos como BRAM) y se verificó en co-simulación con el RTL del chip. Se apoya en tres ideas:

1. **Contar caídas de SCK, no pulsos de subida.** El chip cuenta "ticks" (un ciclo después de cada caída). La n-ésima caída desde que CS_N baja corresponde al tick n-ésimo *aunque se haya perdido el primer bit*. Los 32 bits de comando+dirección se acumulan en un registro de desplazamiento de 32 bits: el bit perdido era siempre un 0, que ya está en el registro, así que `cmd=sh[31:24]` y `addr=sh[23:0]` son correctos en *ambas* alineaciones, sin detectar nada.
2. **Flash: 1 ciclo dummy.** El primer bit de dato se presenta en la caída 33 (no en la 32). El controlador de flash captura entonces 32 bits exactos.
3. **RAM: alineación estándar; la caída coincidente con CS_N no cuenta.** El controlador de la RAM tiene un estado extra (`WAIT_SCLK`); una caída de SCK simultánea con la bajada de CS_N no genera tick, a diferencia de la flash.

Como el emulador muestrea en el flanco de subida del reloj (medio ciclo después de que el chip cambia sus salidas, que cambian en el de bajada), su MISO está estable medio ciclo antes del muestreo del chip (tick = 1 ciclo tras la caída): no hacen falta sincronizadores si se comparte el reloj (unos 20 ns de margen a 24 MHz).

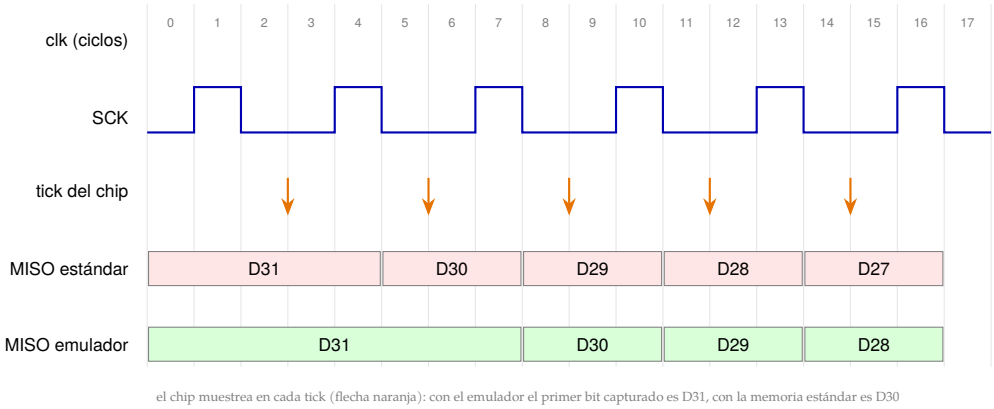


Figura 13: Alineación de datos de flash (periodo SCK = 3 ciclos). El emulador retrasa los datos una caída; el chip muestrea los mismos instantes y captura los 32 bits.

Verificación del emulador	Modelo Python	RTL (co-sim)	Evidencia
Arranque + banner UART	✓	✓	`test_hw1`, REQ-01 (0,69 ms @24 MHz)
Autotest ISA	✓ (8/10)	✓ (8/10)	`ISA PPPPPPPPFF`: los 2 grupos F son DEF-S3
RAM: contenido big-endian y lw 31 bits	✓	✓	`test_hw3`
Barrido de fases flash × RAM (11 desplazamientos)	✓	✓	`test_hw4`
Programa C compilado con GCC	–	✓	`test_hw5`

Cuadro 6: Resultados con el emulador: 5/5 pruebas de co-simulación RTL.

6.3 Alternativas evaluadas

Pre-desplazar la imagen de flash Corrige DEF-F2 pero no DEF-F1 (comando); además no ayuda a la RAM. Insuficiente por sí sola.

Retardar MISO ≈45 ns (RC/buffer) Hace que el chip muestree el bit anterior y arregla los datos de la flash (hasta ≈2 ciclos de margen), pero sigue sin resolver DEF-F1; es un parche frágil dependiente del reloj.

Ajustar la fase por software Los desplazamientos `slli x0,x0,s` permiten cambiar la fase en tiempo de ejecución, pero es un calendario ciclo a ciclo inviable para programas con bucles dependientes de datos o de la UART.

RP2040 de la placa TT Puede emular la memoria si la ruta de respuesta cabe en ≈1 periodo de SCK (125 ns a 8 MHz); el RTL de arriba es la especificación exacta del comportamiento requerido.

6.4 Reglas de firmware (verificadas por las pruebas)

Regla	Qué hacer	Defecto / prueba
R1	**Dirección 0 = NOP.** La instrucción de la dirección 0 nunca se ejecuta; el programa real empieza en 0x4.	DEF-R1, `i00`
R2	**Inicializar todos los registros** a 0 al arrancar (`crt0`): arrancan con basura, igual que `cycles`.	INFO-A3, `f15`
R3	**Código y constantes en la flash** (0x0000–0xFFFF); allí se pueden usar `lb/lh/lw/lbu/lhu` sin restricción.	`f07`, autotest G
R4	**SPI RAM sólo con `lw`/`sw`** alineados; nada de `sb/sh/lb/lh/lbu/lhu` sobre 0x10000+.	DEF-S3
R5	**Valores de 31 bits y máscara:** tras cada `lw` de la RAM hacer `& 0x7FFFFFFF`; no guardar enteros con signo ni punteros $\geq 2^{31}$.	DEF-S2, `f09`
R6	**Evitar la pila (recursión, spills):** el compilador restaura registros sin enmascarar. Mantener el estado en registros.	hw5, §5
R7	**UART:** escribir dato, luego `ctrl=0x08` y volver a `ctrl=0`; esperar `tx_busy` (bit 9) antes de cada carácter.	`i08`, `i09`
R8	**Reloj de placa ≈24 MHz** para 115200 baudios (208 ciclos/bit); no usar 27 MHz. Enviar bytes con una pausa ≥ 1 carácter.	DEF-U1/U2, REQ-05/06

Cuadro 7: Reglas que debe cumplir cualquier firmware para este silicio.

6.5 Compilar C para el chip

```
# verif/firmware/Makefile
CFLAGS  = -march=rv32i -mabi=ilp32 -O2 -ffreestanding -fno-builtin -fno-stack-protector -fomit-frame-pointer
LDFLAGS = -nostdlib -T link.ld -Wl,--gc-sections      # .text+.rodata en la flash @0 ; sin .data/.bss
hello.bin: hello.elf ; riscv64-unknown-elf-objcopy -O binary hello.elf hello.bin
```

`crt0.S` coloca el `nop` sacrificial, pone a cero los registros y fija `sp=0x1FFF0`; `link.ld` descarta `.data/.bss`. Sin extensión M, las multiplicaciones llaman a `__mulsi3` de libgcc (se enlaza con `-lgcc`). El ejemplo `hello.c` (891 bytes) imprime *"C on femto_UN / fib(10)=37 / ram31sum=011D / primes<50=0F"* y enciende el LED; corre en el chip co-simulado con el emulador RTL.

6.6 Lista de verificación para el *bring-up*

1. Reloj de placa ≈24 MHz, el *mismo* que alimenta al emulador. Reset activo en bajo (`rst_n`).
2. Programar el emulador con la imagen (`hello.bin` en el offset 0 de la flash).
3. Observar TXD (uo_out[7]) a 115200 8N1: debe verse el banner a los ≈0,7 ms del reset.
4. Si no aparece nada: medir SCK flash (uo_out[5], $f_{clk}/3$) y confirmar que CS_N baja antes del primer pulso; revisar que reloj y emulador comparten fase.
5. Para depurar: ejecutar `make hw` (co-simulación) con la misma imagen antes de pasarla al hardware.

7 Aceptación de requisitos

Req.	Criterio de aceptación	Mem. comerciales	Con emulador	Medido
REQ-01	Banner UART $\leq$50 000 ciclos tras reset	✗	✓	16 650 ciclos (0,69 ms)
REQ-02	Autotest ISA: 10/10 grupos	✗	✗	8/10 (DEF-S3)
REQ-03	8 palabras de 32 bits en SPI RAM sin error	✗	✗	3/8 incorrectas (DEF-S2)
REQ-04	$\geq$100 kIPS @24 MHz	✗	✓	198 ciclos/instr = 121 kIPS
REQ-05a	Baud TX 115200 $\pm$3 % @24 MHz	✗	✓	+0,16 %
REQ-05b	Baud TX 115200 $\pm$3 % @27 MHz	✗	✗	+12,68 %
REQ-06a	RX de PC @24 MHz con pausa	✗	✓	eco exacto `RV32`
REQ-06b	RX de dos bytes consecutivos	✗	✗	DEF-U2
REQ-06c	RX de PC @27 MHz	✗	✗	DEF-U1
REQ-07	Arranque con 3 estados iniciales aleatorios	✗	✓	3/3
REQ-08	Arranca con flash y RAM comerciales sin lógica extra	✗	✗	comando 0x6 (DEF-F1)

Cuadro 8: Aceptación. "Mem. comerciales" = perfil `standard` (las pruebas de aceptación dependen de que la CPU arranque, por eso todas fallan). REQ-08 se ejecuta siempre con memorias comerciales.

Lectura

Con el emulador se cumplen 5 de 11 criterios; los 6 restantes son limitaciones propias del silicio (31 bits de RAM, sub-palabras, UART a 27 MHz/tramas consecutivas) o la imposibilidad de usar memorias comerciales sin lógica adicional. Ninguno depende de errores del banco de pruebas.

8 Limitaciones del análisis y trabajo futuro

- **Simulación, no silicio:** todo se ejecutó sobre el RTL (sin retardos de celdas ni de pad). El emulador debe validarse en hardware; los márgenes de temporización dados (medio ciclo) son de simulación a reloj común.
- **Modelo FRAM_SPI:** el repositorio no incluye el modelo `FRAM_SPI` del banco original; los esclavos SPI se escribieron según los datasheets (W25Q / 23LC / FM25V). Si el chip de RAM es una FRAM tipo FM25V, exige `WREN` (0x06) antes de escribir y el controlador nunca lo envía; con el emulador no importa.
- **Cobertura de `FENCE`/CSR:** no soportadas por el núcleo; no se probó ningún CSR distinto de `RDCYCLE`.
- **Próximo tape-out (opcional):** los defectos DEF-F1/F2/S1/S2/R1 se corrigen en el RTL; una propuesta verificada (SCK sólo durante la transacción, muestreo en flanco de subida, 32 bits exactos) pasa 12/12 las pruebas unitarias de SPI RAM (`verif/future/MappedSPIRAM_fixed.v`). No se aplicó al silicio.

A Cómo ejecutar la suite

```
# requisitos: iverilog, cocotb>=2.0 (pip install cocotb), riscv64-unknown-elf-gcc (opcional)
cd verif
make unit            # 4 m\'odulos: Quark, SPI RAM, SPI Flash, UART       (53 pruebas)
make integration     # 13     make functional  # 15     make e2e  # 6     make acceptance  # 11
make hw              # co-simulaci\'on con el emulador RTL                  (5 pruebas)
MEM_PROFILE=silicon_aware make all     # emulador (Python) en vez de memorias comerciales
make report          # tabla por nivel; results/<perfil>/summary.json
cd firmware && make  # compila hello.c -> hello.bin (imagen de la flash)
```

Estructura: `lib/` (ensamblador RV32I, ISS, modelos SPI, banco SoC), `unit/`, `integration/`, `functional/`, `e2e/`, `acceptance/`, `hw/` (RTL del emulador y co-simulación), `firmware/`, `tools/` (catálogo de defectos e informe).

B Resultado de cada prueba

Nivel	Prueba	Mem. com.	Emulador	Defecto
U-Quark	reset_and_first_fetch	✓	✓	
U-Quark	lui_auipc	✓	✓	
U-Quark	auipc_addr_width_limitation	✓	✓	
U-Quark	alu_reg_all_ops	✓	✓	
U-Quark	alu_imm_all_ops	✓	✓	
U-Quark	shifts_all_amounts	✓	✓	
U-Quark	shift_rd_equals_rs	✓	✓	
U-Quark	branches_all	✓	✓	
U-Quark	branch_backward_loops	✓	✓	
U-Quark	branch_far_offsets	✓	✓	
U-Quark	jal	✓	✓	
U-Quark	jalr	✓	✓	
U-Quark	load_store_all_widths	✓	✓	
U-Quark	store_widths_and_lanes	✓	✓	
U-Quark	x0_hardwired_zero	✓	✓	
U-Quark	rdcycle_and_system	✓	✓	
U-Quark	cycles_per_instruction	✓	✓	
U-Quark	program_fibonacci	✓	✓	
U-Quark	random_vs_iss_no_wait	✓	✓	
U-Quark	random_vs_iss_with_wait_states	✓	✓	
U-RAM	reset_idle	✓	✓	
U-RAM	sck_timing	✓	✓	
U-RAM	read_protocol_and_data	✗	✗	DEF-S2
U-RAM	write_protocol_and_data	✓	✓	
U-RAM	write_then_read_roundtrip	✗	✗	DEF-S1
U-RAM	adjacent_words_do_not_overlap	✗	✗	DEF-S1, DEF-S2
U-RAM	read_all_divider_phases	✗	✗	DEF-S1, DEF-S2
U-RAM	write_all_divider_phases	✗	✓	DEF-S1
U-RAM	spi_signal_timing	✗	✗	DEF-C1
U-RAM	random_mixed_traffic	✗	✗	DEF-S1, DEF-S2
U-RAM	miso_output_delay_tolerance	✗	✗	DEF-S2
U-RAM	reset_during_transaction	✓	✓	
U-Flash	reset_idle	✓	✓	
U-Flash	sck_period_and_duty	✓	✓	
U-Flash	read_protocol_and_data	✗	✗	DEF-F2
U-Flash	byte_order_little_endian	✗	✓	DEF-F2
U-Flash	sequential_reads_distinct_words	✗	✓	DEF-F2
U-Flash	command_and_address_integrity_all_phases	✗	✓	DEF-F1
U-Flash	data_integrity_all_phases	✗	✓	DEF-F1, DEF-F2
U-Flash	address_range	✗	✓	DEF-F2
U-Flash	busy_handshake	✓	✓	
U-Flash	spi_signal_timing	✗	✗	DEF-C1
U-Flash	miso_output_delay_tolerance	✗	✓	DEF-F2
U-Flash	reset_during_transaction	✗	✓	DEF-F2
U-UART	reset_state	✓	✓	
U-UART	register_map_and_led	✓	✓	
U-UART	tx_frame_format	✓	✓	
U-UART	tx_baud_rate_vs_spec	✗	✗	DEF-U1
U-UART	tx_busy_and_ignored_write	✓	✓	
U-UART	rx_at_own_timebase	✓	✓	
U-UART	rx_from_ideal_115200_sender	✗	✗	DEF-U1
U-UART	rx_framing_error	✗	✗	DEF-U2
U-UART	rx_back_to_back_frames	✗	✗	DEF-U2
Integ.	i01_first_fetch_comes_from_flash_at_reset_address	✗[b]	✓	
Integ.	i00_first_instruction_after_reset_is_executed	✗	✗	DEF-R1
Integ.	i02_instruction_stream_integrity	✗[b]	✓	
Integ.	i03_address_decoder_map	✗[b]	✓	
Integ.	i04_cpu_spiram_word_roundtrip	✗[b]	✓	
Integ.	i05_cpu_spiram_full_32bit_words	✗[b]	✗	DEF-S2
Integ.	i06_cpu_spiram_byte_halfword_access	✗[b]	✗	DEF-S3
Integ.	i07_busy_handshake_stalls_cpu	✗[b]	✓	
Integ.	i08_uart_write_path	✗[b]	✓	
Integ.	i09_uart_status_read_path	✗[b]	✓	
Integ.	i10_no_crosstalk_between_flash_and_ram	✗[b]	✓	
Integ.	i11_store_to_flash_space_is_ignored	✗[b]	✓	
Integ.	i12_unmapped_addresses_alias_to_flash	✗[b]	✓	
Func.	f01_alu_register_ops	✗[b]	✓	
Func.	f02_alu_immediate_ops	✗[b]	✓	
Func.	f03_shifts_all_forms	✗[b]	✓	
Func.	f04_branches_taken_and_not_taken	✗[b]	✓	
Func.	f05_jal_jalr_call_return	✗[b]	✓	
Func.	f06_lui_auipc	✗[b]	✓	
Func.	f07_loads_from_flash_all_widths	✗[b]	✓	
Func.	f08_stack_recursion_in_spiram	✗[b]	✓	
Func.	f09_ram_word_store_load_masked31	✗[b]	✓	
Func.	f10_ram_subword_store_load	✗[b]	✗	DEF-S3
Func.	f11_uart_transmit_char	✗[b]	✓	
Func.	f12_uart_receive_byte	✗[b]	✓	
Func.	f13_led_control	✗[b]	✓	
Func.	f14_rdcycle_monotonic_and_latency	✗[b]	✓	
Func.	f15_power_up_state_is_not_zero	✗[b]	✓	
E2E	e01_hello_world	✗[b]	✓	
E2E	e02_compute_and_print_fibonacci	✗[b]	✓	

Nivel	Prueba	Mem. com.	Emulador	Defecto
E2E	`e03_uart_echo`	✗^{b}	✓	
E2E	`e04_led_blink`	✗^{b}	✓	
E2E	`e05_reset_restarts_program`	✗^{b}	✓	
E2E	`e06_isa_selftest`	✗^{b}	✗	DEF-S3
Acept.	`req01_boot_banner_within_budget`	✗^{b}	✓	
Acept.	`req02_isa_selfall_groups_pass`	✗^{b}	✗	DEF-S3
Acept.	`req03_spiram_32bit_data_integrity`	✗^{b}	✗	DEF-S2
Acept.	`req04_cpu_throughput`	✗^{b}	✓	
Acept.	`req05a_uart_tx_baud_at_24mhz`	✗^{b}	✓	
Acept.	`req05b_uart_tx_baud_at_27mhz`	✗^{b}	✗	DEF-U1
Acept.	`req06a_uart_rx_from_pc_24mhz_spaced`	✗^{b}	✓	
Acept.	`req06b_uart_rx_back_to_back`	✗^{b}	✗	DEF-U2
Acept.	`req06c_uart_rx_from_pc_27mhz`	✗^{b}	✗	DEF-U1
Acept.	`req07_power_up_robustness`	✗^{b}	✓	
Acept.	`req08_commercial_spi_memories`	✗^{b}	✗^{b}	DEF-F1, DEF-F2
HW	`hw1_hello_with_rtl_memory_emulator`	–	✓	
HW	`hw2_isa_selfwith_rtl_memory_emulator`	–	✓	
HW	`hw3_ram_contents_big_endian_in_emulator`	–	✓	
HW	`hw4_phase_sweep_flash_and_ram`	–	✓	
HW	`hw5_compiled_c_firmware`	–	✓	

✓ pasa, ✗ falla; ^{b} = bloqueada porque la CPU no arranca (“la CPU no arranca” en el mensaje del fallo). Las pruebas HW se ejecutan sólo con el emulador RTL (— = no aplica).

C RTL del emulador de memoria

```
// SPDX-License-Identifier: Apache-2.0
// silicon_spi_mem.v -- Memoria SPI "consciente del silicio" (flash + RAM) para FPGA/CPLD.
//
// Sustituye a la flash y a la RAM SPI de la placa del chip femto_UN (commit f15fccd).
// Conoce los defectos del silicio y los compensa del lado de la memoria:
//   * Cuenta flancos de BAJADA de SCK desde que CS_N baja (no pulsos de subida). Así no le afecta que el
//     chip pierda el primer bit del comando (DEF-F1/DEF-S1): comando y dirección son siempre los 32 bits
//     de sh[31:0] (el bit perdido siempre es un 0 y entra como 0 al registro de desplazamiento).
//   * Flash: el primer bit de dato se presenta en la caída nº 33 (equivale a 1 ciclo "dummy"): el controlador
//     de flash del chip captura entonces los 32 bits exactos (corrige DEF-F2).
//   * RAM: el controlador captura 31 bits (DEF-S2): D31 (caída 32) lo ignora el chip; se presenta D30 en la caída 33;
//     el firmware debe tratar las palabras de RAM como de 31 bits.
//   * La flash cuenta como "tick" la caída de SCK coincidente con la bajada de CS_N; la RAM no (la RAM tiene
//     un estado extra WAIT_SCLK en su controlador).
// Debe usar el MISMO reloj `clk` que el chip: las salidas del chip cambian en el flanco de bajada, el emulador
// las muestrea en el de subida (medio ciclo después) y su MISO llega antes del siguiente muestreo del chip.
module silicon_spi_mem #(
    parameter FLASH_AW = 14,          // 2**14 = 16 KiB de flash
    parameter RAM_AW   = 16           // 64 KiB de RAM (el chip direcciona 16 bits)
) (
    input  wire clk,
    input  wire f_sck, f_csn, f_mosi, output reg f_miso,
    input  wire r_sck, r_csn, r_mosi, output reg r_miso
);
    // ---------------------------------------------------------------- FLASH (sólo lectura, 0x03)
    reg [7:0] fmem [0:(1<<FLASH_AW)-1];
    reg       f_sck_q = 1'b0, f_cs_q = 1'b1;
    reg [6:0]  f_nf;
    reg [31:0] f_sh;
    reg [23:0] f_base;
    reg       f_ok;
    reg [7:0]  f_byte;

    wire f_act    = ~f_csn;
    wire f_start  = f_cs_q & ~f_csn;                    // CS_N acaba de bajar
    wire f_rise   = ~f_sck_q & f_sck & f_act;
    wire f_fall   = f_sck_q & ~f_sck & f_act;
    wire [6:0]  f_nf_cur = f_start ? 7'd0 : f_nf;
    wire [31:0] f_sh_cur = f_start ? 32'd0 : f_sh;
    wire [6:0]  f_nf_new = f_nf_cur + {6'd0, f_fall};   // la caída coincidente con CS_N SÍ cuenta
    wire [6:0]  f_k      = f_nf_new - 7'd33;            // índice del bit de dato que toca presentar
    // dirección del byte que se necesitará en la PRÓXIMA caída (índice f_nf+1-33), para que f_byte esté listo a tiempo
    wire [6:0]  f_k_next = f_nf - 7'd32;
    wire [FLASH_AW-1:0] f_raddr = f_base[FLASH_AW-1:0] + {{(FLASH_AW-3){1'b0}}, f_k_next[6:3]};

    always @(posedge clk) begin
        f_sck_q <= f_sck;
        f_cs_q  <= f_csn;
        f_byte  <= fmem[f_raddr];                       // lectura síncrona (BRAM)
        if (f_csn) begin
            f_nf <= 0; f_sh <= 0; f_ok <= 0; f_miso <= 1'b0;
        end else begin
            f_nf <= f_nf_new;
            f_sh <= f_rise ? {f_sh_cur[30:0], f_mosi} : f_sh_cur;
            if (f_fall && f_nf_new == 7'd32) begin      // dirección completa (en cualquier alineación)
                f_base <= f_sh_cur[23:0];
                f_ok   <= (f_sh_cur[31:24] == 8'h03);
            end
            if (f_fall && f_nf_new >= 7'd33 && f_ok)
                f_miso <= f_byte[3'd7 - f_k[2:0]];
        end
    end

    // ---------------------------------------------------------------- RAM (lectura 0x03 / escritura 0x02)
    reg [7:0] rmem [0:(1<<RAM_AW)-1];
    reg       r_sck_q = 1'b0, r_cs_q = 1'b1;
    reg [6:0]  r_nf;
    reg [31:0] r_sh;
    reg [23:0] r_base;
    reg [1:0]  r_mode;                                  // 0 nada, 1 lectura, 2 escritura
    reg [6:0]  r_wcnt;
```

```
    reg [7:0]  r_wsh;
    reg [7:0]  r_byte;

    wire r_act   = ~r_csn;
    wire r_start = r_cs_q & ~r_csn;
    wire r_rise  = ~r_sck_q & r_sck & r_act;
    wire r_fall  = r_sck_q & ~r_sck & r_act;
    wire r_fall_cnt = r_fall & ~r_start;                  // aquí la caída coincidente NO cuenta
    wire [6:0]  r_nf_cur = r_start ? 7'd0 : r_nf;
    wire [31:0] r_sh_cur = r_start ? 32'd0 : r_sh;
    wire [6:0]  r_nf_new = r_nf_cur + {6'd0, r_fall_cnt};
    wire [6:0]  r_k      = r_nf_new - 7'd32;
    wire [6:0]  r_k_next = r_nf - 7'd31;                   // índice del bit de la próxima caída (r_nf+1-32)
    wire [RAM_AW-1:0] r_raddr = r_base[RAM_AW-1:0] + {{(RAM_AW-3){1'b0}}, r_k_next[6:3]};

    always @(posedge clk) begin
        r_sck_q <= r_sck;
        r_cs_q  <= r_csn;
        r_byte  <= rmem[r_raddr];
        if (r_csn) begin
            r_nf <= 0; r_sh <= 0; r_mode <= 0; r_miso <= 1'b0; r_wcnt <= 0; r_wsh <= 0;
        end else begin
            r_nf <= r_nf_new;
            r_sh <= (r_rise && r_mode == 0) ? {r_sh_cur[30:0], r_mosi} : r_sh_cur;
            if (r_fall_cnt && r_nf_new == 7'd32) begin
                r_base <= r_sh_cur[23:0];
                r_mode <= (r_sh_cur[31:24] == 8'h03) ? 2'd1 : (r_sh_cur[31:24] == 8'h02) ? 2'd2 : 2'd0;
            end
            if (r_fall_cnt && r_nf_new >= 7'd33 && r_mode == 2'd1)
                r_miso <= r_byte[3'd7 - r_k[2:0]];
            if (r_rise && r_mode == 2'd2) begin             // datos de escritura: MSB primero
                r_wsh   <= {r_wsh[6:0], r_mosi};
                r_wcnt  <= r_wcnt + 7'd1;
                if (r_wcnt[2:0] == 3'd7)
                    rmem[r_base[RAM_AW-1:0] + {{(RAM_AW-3){1'b0}}, r_wcnt[6:3]}] <= {r_wsh[6:0], r_mosi};
            end
        end
    end
endmodule
```

D Arranque y ejemplo en C

```
/* crt0.S -- arranque para el silicio femto_UN (commit f15fccd)
 *
 * Reglas de silicio aplicadas aquí:
 *  R1  La instrucción de la dirección 0 NUNCA se ejecuta (la 1ª lectura tras reset se pierde, DEF-R1):
 *      se coloca un NOP sacrificial.
 *  R2  Los registros arrancan con basura: se ponen todos a 0 antes de usarlos.
 *  R3  La pila va en la SPI RAM (0x10000..0x1FFFF), sólo con accesos de palabra (lw/sw).
 */
    .section .init, "ax"
    .globl _start
_start:
    nop                         /* R1: dirección 0, se pierde */
    li   x1,  0                 /* R2 */
    li   x3,  0
    li   x4,  0
    li   x5,  0
    li   x6,  0
    li   x7,  0
    li   x8,  0
    li   x9,  0
    li   x10, 0
    li   x11, 0
    li   x12, 0
    li   x13, 0
    li   x14, 0
    li   x15, 0
    li   x16, 0
    li   x17, 0
    li   x18, 0
    li   x19, 0
    li   x20, 0
    li   x21, 0
    li   x22, 0
    li   x23, 0
    li   x24, 0
    li   x25, 0
    li   x26, 0
    li   x27, 0
    li   x28, 0
    li   x29, 0
    li   x30, 0
    li   x31, 0
    lui  sp, 0x20               /* R3: sp = 0x20000 (fin de la SPI RAM) */
    addi sp, sp, -16
    call main
1:  j    1b
```

```
/* hello.c -- C sobre femto_UN respetando las reglas de silicio
  *   - sin variables globales (no hay .data/.bss) y SIN pila útil: todo el estado vive en registros
 *   - la SPI RAM sólo con ram_store/ram_load (palabras de 31 bits, máscara del bit 31, DEF-S2)
 *   - sin M (no hay mul/div): se usan sumas y restas
 *   - constantes (cadenas) en la flash, leídas con lbu (la flash sí admite todos los anchos)
 */
#include <stdint.h>
#define UART_DATA (*(volatile uint32_t *)0x400008)
#define UART_CTRL (*(volatile uint32_t *)0x400010)
#define LED_BIT   0x4

static void putc_(char c) {
    while (UART_CTRL & 0x200) ;      /* tx_busy */
    UART_DATA = (uint32_t)c;
    UART_CTRL = 0x8;                  /* tx_wr = 1 */
    UART_CTRL = 0x0;                  /* hay que bajarlo: es un nivel, no un pulso */
}
static void puts_(const char *s) { while (*s) putc_(*s++); }
static void puthex(uint32_t v, int n) {
```

```
    for (int i = n - 1; i >= 0; i--) {
        uint32_t d = (v >> (4 * i)) & 15;
        putc_(d < 10 ? '0' + d : 'A' + d - 10);
    }
}
/* SIN recursión: GCC guardaría registros en la pila (SPI RAM) y al restaurarlos el bit 31 llega aleatorio (DEF-S2). */
static uint32_t fib(uint32_t n) { uint32_t a = 0, b = 1; while (n--) { uint32_t t = a + b; a = b; b = t; } return a; }

/* Acceso EXPLÍCITO a la SPI RAM: sólo lw/sw, valores de 31 bits y máscara del bit 31 tras cada lectura. */
#define RAM ((volatile uint32_t *)0x10000)
static inline void     ram_store(uint32_t i, uint32_t v) { RAM[i] = v & 0x7FFFFFFF; }
static inline uint32_t ram_load(uint32_t i)              { return RAM[i] & 0x7FFFFFFF; }

int main(void) {
    puts_("C on femto_UN\n");
    puts_("fib(10)=");  puthex(fib(10), 2); putc_('\n');
    for (uint32_t i = 0; i < 10; i++) ram_store(i, i * i);          /* tabla de cuadrados en la SPI RAM */
    uint32_t sum = 0; for (uint32_t i = 0; i < 10; i++) sum += ram_load(i);
    puts_("ram31sum="); puthex(sum, 4); putc_('\n');                /* 285 = 0x011D */
    uint32_t primes = 0;                                    /* primos < 50 por división repetida */
    for (uint32_t n = 2; n < 50; n++) {
        uint32_t is_p = 1;
        for (uint32_t d = 2; d * d <= n; d++) {             /* d*d con M ausente -> __mulsi3 */
            uint32_t m = n; while (m >= d) m -= d;
            if (m == 0) { is_p = 0; break; }
        }
        primes += is_p;
    }
    puts_("primes<50="); puthex(primes, 2); putc_('\n');
    UART_CTRL = LED_BIT;                                    /* enciende el LED */
    for (;;) ;
}
```